

REPRODUCIBLE MATHEMATICA AND LEAN VERIFICATION FOR TRACE-EUCLIDEAN-V9

COMPUTATIONAL COMPANION AND MAINTENANCE GUIDE

ABSTRACT. This document records the reproducible Mathematica checks and selected Lean 4 proof obligations associated with `Trace-Euclidean-v9.tex`. It records the SHA-256 digest of the private manuscript version used to prepare the extracted inputs, maps each mathematical component to a named code module and stable internal LaTeX anchor, and imports all numerical results from machine-readable output. In the pages below, those anchors are rendered as the theorem, equation, section, and table numbers in the compiled manuscript, together with page numbers where they help direct comparison. Exact identities, certified interval calculations, finite enumerations, and finite diagnostics are kept distinct. The accompanying workflow is designed so that a later manuscript revision requires changes only in the affected manifest entry or check module; the result tables in this document are then regenerated automatically. The public repository intentionally excludes both the manuscript TeX source and the rendered manuscript PDF.

1. PURPOSE AND EVIDENCE BOUNDARY

The package checks the formulas and finite computations used in the manuscript. A record marked **PASS** establishes only the scope stated for that check. In particular, a finite diagnostic does not prove a theorem for all number fields or all lattice vectors, and a high-precision root approximation does not certify existence or uniqueness. General compactness, monotonicity, finiteness, and source-dependent assertions remain mathematical arguments in the manuscript.

The evidence classes used here are:

- (i) an *exact symbolic identity*, simplified with exact input;
- (ii) a *certified rational interval*, whose endpoints are exact rational numbers and whose sign excludes zero;
- (iii) a *finite enumeration*, over the complete rectangle recorded below;
- (iv) a *high-precision approximation*, used only to check printed decimals;
- (v) a *finite diagnostic*, explicitly limited to the configured sample.

Publication boundary. The repository-level publication check permits only explicitly listed verification-document `.tex` and `.pdf` files. It rejects the private source-snapshot and manuscript-build directories, complete TeX or PDF documents outside that allowlist, and opaque archives. Consequently, the recorded source digest identifies the manuscript version used for extraction without distributing the manuscript source or its rendered PDF.

2. VERSION BINDING AND RECORDED RUN

The private maintainer extraction step computes the SHA-256 digest of the manuscript snapshot. During that build, the Mathematica driver recomputes the digest before loading any mathematical module, so a changed snapshot cannot silently reuse stale inputs. The public repository does not contain that snapshot. A public run therefore reads the

recorded digest from `inputs/manuscript_inputs.json`, checks its form and agreement with the result metadata, and then reruns the published calculations. This permits independent reproduction of the computations without distributing the manuscript itself.

Item	Recorded value
Private source identifier	<code>Trace-Euclidean-v9.tex</code>
SHA-256	<code>a2f522d077c600e0dc747dcaa8b06ba4e31ecd0b49b83dae468c9b61915a0e99</code>
Source lines	2372
Wolfram kernel	15.0.0 for Microsoft Windows (64-bit) (May 15, 2026)
PASS / WARN / FAIL	2165 / 0 / 0
Certified classical pairs	24
Certified integral pairs	112
Square-free parameters tested	121
Root precision	80 digits
Rational interval output	40 decimal places, rounded outward

The status convention is as follows. A **FAIL** is a failed exact, certified, coverage, or integrity check and gives batch exit code 1. A **WARN** records a manuscript presentation or rounding issue while the verification engine remains internally consistent and gives exit code 2. A run with neither gives exit code 0.

3. SOURCE, CODE, AND RESULT CORRESPONDENCE

Every test record in `results/tests.json` and `results/tests.csv` contains the fields `id`, `module`, `anchor`, `evidence`, `category`, `status`, `actual`, `expected`, and `note`. Thus a result can be traced in both directions: from a manuscript label to a code module and its test records, or from a test ID back to the relevant manuscript location. Stable LaTeX labels remain internal machine keys; the manual displays the corresponding compiled references, such as “Lemma 4.6,” rather than raw label names. Table checks additionally use stable row IDs such as `admissible-classical:r01`; source line numbers are retained only as a convenience and are not used as identifiers.

Analytic functions, derivatives, and extrema

Code: `checks/analytic_bounds.wl`. **Result (P/W/F):** 134 / 0 / 0.

Manuscript locations: Equation (4.1), Lemma 4.1, Lemma 4.2, Lemma 4.3, Lemma 4.4, Lemma 4.5, Lemma 4.6, Lemma 4.7, Lemma 4.8, Lemma 4.9.

Evidence: exact symbolic identities, certified rational intervals, and high-precision approximations.

Admissible-pair tables and excluded tails

Code: `checks/admissible_tables.wl`. **Result (P/W/F):** 1677 / 0 / 0.

Manuscript locations: Table 1, Table 2, Table 3, Table 4.

Evidence: certified finite enumeration plus analytic tail conditions from the manuscript.

Trace Gram matrices and quadratic-field Voronoi cells

Code: `checks/trace_geometry.wl`. **Result (P/W/F):** 272 / 0 / 0.

Manuscript locations: Lemma 3.1, Proposition 6.1, Equation (6.2), Theorem 1.8.

Evidence: exact algebra, exact Voronoi vertices, and bounded finite diagnostics.

Gamma-function boundary diagnostics

Code: `checks/gamma_diagnostics.wl`. **Result (P/W/F):** 81 / 0 / 0.

Manuscript locations: Equation (3.12), Equation (3.13).

Evidence: finite high-precision diagnostics; the cited analytic inequalities remain external inputs.

The machine-readable version of this table is `results/verification_ledger.json`. The detailed result count by check category is:

Check category	Total	PASS	WARN	FAIL
Gamma finite sample	81	81	0	0
certified table upper bound	42	42	0	0
exact Voronoi	13	13	0	0
exact algebra	58	58	0	0
exact classification	1	1	0	0
exact implication	2	2	0	0
exact inequality	5	5	0	0
exact integer proof	2	2	0	0
exact trace Gram	8	8	0	0
finite diagnostic	121	121	0	0
finite enumeration	1525	1525	0	0
finite field verification	121	121	0	0
integrity	1	1	0	0
manuscript approximation	15	15	0	0
rational certificate	46	46	0	0
source binding	9	9	0	0
symbolic limit	3	3	0	0
symbolic sign	10	10	0	0
table approximation	34	34	0	0
table coverage	2	2	0	0
table display precision	34	34	0	0
table maxima	24	24	0	0
table outward interval	8	8	0	0

4. ANALYTIC DEFINITIONS AND EXTREMA

The code uses the manuscript definitions

$$\begin{aligned}
 a &= \frac{1}{6}, & b &= \frac{e^2}{2\pi} - 1, \\
 K(x) &= \log(2\pi) - \log x - 1, & g(x, y) &= x \log(2\pi(y + b)) - \log(\pi(xy + 2a)) + xyK(x), \\
 g_s(x, y) &= \frac{g(x, y)}{x}, & g_n(x, y) &= y \log 2 + \frac{g(x, y)}{x}.
 \end{aligned}$$

The module first compares exact symbolic derivatives with the displayed formulas. It then checks the signs needed at the stated thresholds by rational interval arithmetic. The roots returned by `FindRoot` at 80-digit working precision are used only for the following comparison with decimals printed in the manuscript.

Claim ID	Manuscript location	Line	Printed	Computed	Status
b	Section 4	1098	0.176	0.1760048029	PASS
y0	Lemma 4.1	1128	25.194	25.1938160912	PASS

Claim ID	Manuscript location	Line	Printed	Computed	Status
y1	Lemma 4.1	1164	5.7	5.6579939435	PASS
x0	Lemma 4.2	1174	4.375	4.3745521287	PASS
z-minus	Lemma 4.2	1200	0.477	0.4765865552	PASS
z-plus	Lemma 4.2	1200	0.699	0.6994182477	PASS
g-maximum	Lemma 4.3	1247	3.329	3.3286422771	PASS
x-boundary	Lemma 4.3	1249	5.253	5.2534385501	PASS
y-rank3	Lemma 4.6	1365	2.347	2.3471186383	PASS
gs-max	Lemma 4.6	1363	1.104	1.103803611	PASS
two-pi-over-e	Lemma 4.6	1375	2.311	2.3114546996	PASS
xn	Lemma 4.8	1391	9.803	9.8031890569	PASS
gn-max	Lemma 4.9	1460	2.361	2.3610934112	PASS
yn-rank5	Lemma 4.9	1462	9.999	9.9985918124	PASS
four-pi-over-e	Lemma 4.9	1470	4.623	4.6229093992	PASS

For the global comparison in Lemma 4.3, the computation certifies

$$2.346 < y_0(3) < 2.348, \quad 5.252 < x_0(1) < 5.254, \\ g(3, y_0(3)) < 3.325 < 3.328 < g(5.253, 1).$$

The Hessian identities are checked symbolically. The argument that the maximum is attained and that no point at infinity is omitted remains the analytic argument in the manuscript.

5. COMPLETE INVENTORY OF SECTION 4 CALCULATIONS

The following ledger lists every calculation in Section 4 that can be checked directly by the present Mathematica methods: definitions and symbolic derivatives, algebraic reductions, interval-certified signs and root brackets, threshold evaluations, Hessian tests, boundary comparisons, and the stated one-variable limits. Each row names the exact location in the compiled manuscript and the evidence used by its associated test records.

Manuscript location	Calculation checked	Evidence	Result
Section 4, p. 13	The printed value of $b=e^2/(2\pi)-1$.	80-digit evaluation and comparison with the printed decimal	PASS (1)
Equation (4.1), p. 14	The formulas for g , g_s , and g_n , all five displayed first and second partial derivatives, and the substitution leading to G_t .	exact symbolic identities plus rational certificates for $0 < b < 1/4 < 2a$	PASS (12)
Lemma 4.1, p. 14	The factorization, intermediate square identity, positive coefficient, and strict negativity of the second x -derivative of g .	exact symbolic identities and quantified sign check	PASS (4)
Lemma 4.1, p. 14	The definition of $f(y)=\partial_x g(1,y)$, its first two derivatives, and the expression of f'' through h .	exact symbolic identities	PASS (4)
Lemma 4.1, p. 14	The derivatives of h , the three displayed constant identities, and positivity of $h'(1)$ and $h(1)$.	exact identities, quantified sign check, and outward rational intervals	PASS (9)

Manuscript location	Calculation checked	Evidence	Result
Lemma 4.1, p. 14	The displayed values $f'(1)$ and $f(1)$, their signs, the limiting sign of f' , and the limits of f and $\text{partial}_x g$.	exact identities, symbolic limits, and outward rational intervals	PASS (7)
Lemma 4.1, p. 14	The roots y_1 and y_0 , including strict rational brackets and the decimals printed in the manuscript.	outward rational root brackets and 80-digit approximations	PASS (6)
Lemma 4.2, p. 15	The formula $h=\text{partial}_y g/x$, its y -derivative, its strict negative sign, and its limit $K(x)$.	exact identities, quantified sign check, and symbolic limit	PASS (4)
Lemma 4.2, p. 15	The critical-point formula after $x=z^2$, its equivalent quadratic inequality, and the two printed quadratic roots below 1.	exact identities, rational endpoint sign, and 80-digit approximations	PASS (7)
Lemma 4.2, p. 15	The threshold $K(2\pi/e)=0$, the inequality $2\pi/e>1$, monotonicity of K , and positivity of the reciprocal difference used in case (i).	exact identities, quantified sign check, and outward rational interval	PASS (5)
Lemma 4.2, p. 15	The definition and derivative of $f(x)=h(x,1)$, its strict decrease and limit, the positivity at $2\pi/e$, and the unique threshold x_0 .	exact identities, symbolic sign and limit, rational root bracket, and 80-digit approximation	PASS (8)
Lemma 4.3, p. 16	The critical-point identity, strict negativity of f' , both differentiated identities at a critical point, and the deduction $g_{xy}<0$ from $g_{xx}<0$.	exact symbolic identities, quantified polynomial sign, and a quantified real implication	PASS (6)
Lemma 4.3, p. 16	The Hessian determinant reduction and the strict negative sign at every interior critical point.	exact symbolic reduction and a quantified real implication	PASS (2)
Lemma 4.3, p. 16	Both boundary equations, their root brackets, and the certified strict comparison selecting $g(x_0(1),1)$.	exact identities, outward rational intervals, and printed-value checks	PASS (11)
Lemma 4.3, p. 16	The two tail limits, $K(3)<0$, and the monotonic decrease of $xK(x)$ for $x\geq 3$ used to obtain a uniform y -tail bound.	exact symbolic limits and derivatives, quantified sign, and outward rational intervals	PASS (7)
Lemma 4.4, p. 17	The x -derivative of g_s , the reduction to $f_s(z)$, the derivative of f_s , $z\geq 4/3$, positivity of $3z^2-3z+1$, and $f_s(4/3)<0$.	exact identities, quantified inequalities, and outward rational interval	PASS (6)
Lemma 4.5, p. 18	The identity $\text{partial}_y g_s=(\text{partial}_y g)/x$ transferring the signs and zeros from Lemma 4.2.	exact symbolic identity	PASS (1)
Lemma 4.6, p. 18	The ordering $2\pi/e<3<x_0$ needed at $x=3$, together with the printed boundary point and maximum of g_s .	outward rational interval and 80-digit approximations	PASS (6)
Lemma 4.7, p. 18	The identity $\text{partial}_x g_n=\text{partial}_x g_s$ transferring strict decrease in x .	exact symbolic identity	PASS (1)
Lemma 4.8, p. 18	The first and second y -derivatives of g_n and the limiting value $\log(4\pi/(xe))$.	exact symbolic identities and symbolic limit	PASS (3)
Lemma 4.8, p. 18	The definition and derivative of $f_n(x)$, strict decrease and limit, positivity at $4\pi/e$, and the unique threshold x_n .	exact identities, symbolic sign and limit, rational root bracket, and 80-digit approximation	PASS (9)

Manuscript location	Calculation checked	Evidence	Result
Lemma 4.9, p. 20	The ordering $4\pi/e < 5 < x_n$, the root $y_n(5)$, and the printed maximum of g_n .	outward rational intervals and 80-digit approximations	PASS (7)

This is a computational coverage statement, not a replacement for the proof. Arguments that pass from these calculations to global existence, uniqueness, or exhaustion of all cases still depend on the hypotheses and deductions in the manuscript. The ledger makes that boundary visible while ensuring that no computational test in the Section 4 analytic module is left unassigned.

6. CERTIFIED INTERVAL CONSTRUCTION

The interval implementation uses exact rational endpoints. It bounds π through Machin's identity

$$\pi = 16 \arctan\left(\frac{1}{5}\right) - 4 \arctan\left(\frac{1}{239}\right)$$

and an alternating arctangent series through exponent 121. It bounds e by the sum through $1/100!$ together with an explicit geometric remainder. For a positive rational number, powers of 2 first reduce the logarithm argument to $[1, 2]$; with $u = (r-1)/(r+1)$, the code uses

$$\log r = 2 \sum_{j=0}^{47} \frac{u^{2j+1}}{2j+1} + R, \quad 0 \leq R \leq \frac{2u^{97}}{97(1-u^2)}.$$

Every arithmetic operation is rounded outward to 40 decimal places, while the stored endpoints remain rational numbers. Consequently, a certified positive or negative sign does not depend on floating-point evaluation.

7. ADMISSIBLE-PAIR TABLES

For the classical case the complete rectangle is $3 \leq n \leq 12$, $1 \leq d \leq 10$; for the integral case it is $5 \leq n \leq 29$, $1 \leq d \leq 57$. The computation classifies every integer point by a rational interval for $g_s(n, d)$ or $g_n(n, d)$, and compares the positive set with the rows extracted from the two admissible-pair tables. The tail signs at $(13, 1)$, $(3, 11)$, $(30, 1)$, and $(5, 58)$ are certified separately. Exhaustion outside the two rectangles also uses the monotonicity lemmas in the manuscript.

For a table range, its lower endpoint is required to be rounded downward and its upper endpoint upward. A number described as an upper bound is checked by certifying $g(n, d) < \log B$. The current run produced the following records that are not **PASS**:

No **WARN** or **FAIL** record was produced by this run.

If a future row is reported here, its stable row ID identifies the extracted table entry. For a rounding warning, the final column gives the smallest decimal at the same precision that the interval computation certifies as an outward upper bound. The present values in Table 1 have been adjusted by this rule.

8. TRACE GRAM AND VORONOI CALCULATIONS

The trace-Gram module tests non-diagonal binary Gram matrices together with distinct fractional principal ideals over several real quadratic fields. All entries and determinants are reduced exactly modulo $z^2 - m$. These tests check the determinant formula on deliberately nontrivial inputs; the general formula remains the symbolic proof in Lemma 3.1.

For the hexagonal case, the code solves the adjacent bisector equations exactly and verifies all six displayed facet inequalities. If

$$G = \begin{pmatrix} p & r \\ r & q \end{pmatrix},$$

then the computed vertex norm is simplified to

$$\frac{pq(p + q - 2r)}{4(pq - r^2)}.$$

After substituting the real-quadratic Gram matrix, each vertex has squared norm $(m + 1)^2/(8m)$. The additional enumeration over square-free $2 \leq m \leq 200$ is a finite diagnostic. The Lean proof establishes the covering upper bound for every point of the real plane and proves sharpness at an explicit deep hole, so it does not depend on this finite enumeration.

9. LEAN FORMALIZATION AND SEMANTIC BOUNDARY

The directory `lean/` is a Lake project pinned to Lean 4.32.1 and a fixed mathlib revision. In addition to the original component checks, it now defines the manuscript’s number-field lattice semantics and actual quotient; constructs pseudobases for nonfree projective lattices; proves the general trace determinant, covolume, and discriminant bounds; constructs finite fixed-field reduction codes; proves exact analytic tails and rank-degree envelopes; applies mathlib’s Hermite theorem; proves exact covering radii on the full real coordinate plane; constructs both rings of integers of $\mathbb{Q}(\sqrt{m})$; and proves the complete concrete classification

$$\text{2-trace Euclidean} \iff m \in \{2, 5, 13\}.$$

The formal source contains no `sorry`, `sorryAx`, `project axiom`, `native_decide`, `run_tac`, `unsafe`, `extern`, or `implemented_by`. The public endpoint audit prints only `propext`, `Classical.choice`, and `Quot.sound`. These are standard Lean and mathlib logical dependencies.

The public audit covers 50 endpoints. The quadratic classification, both covering-radius cases, and all eight conclusions of Theorems 1.2 and 1.3 are concrete endpoints. The finiteness statements quantify over the actual equivalence classes of field–lattice pairs and contain no project-specific proof-obligation argument. The package therefore reaches Grade B under the semantic rubric: the core results are substantially faithful and have complete Lean proofs, while the listed author/domain and independent Lean confirmation items remain unsigned. The exact paper-to-code map and the English semantic-fidelity audit are in `THEOREM_INDEX.md` and `docs/audit/`.

10. REPRODUCTION

The public code, extracted inputs, results, and documentation are available at <https://github.com/hzldew-git/PaperVerification/tree/main/Trace-Euclidean>. A detached clean worktree based on code-and-tooling commit `ec66708818e971b349e71777327ab638b3fa4554` passed the complete public protocol below. All nine local dependency revisions matched the committed manifest, and the verification PDF had the same SHA-256 digest in both checkout paths. The final evidence-only update was rerun in that

worktree and remained clean afterward. To rerun the published calculations from the package root, use:

```
wolframscript -file run_verification.wls
python tools/build_report.py
.\tools\build_verification_manual.ps1
python tools/check_delivery.py
```

For the formalized components, change to `lean/` and run:

```
lake exe cache get
lake build
lake env lean TraceEuclideanTest/MainTheoremAudit.lean
```

No manuscript file is required for these commands. To inspect variables interactively, evaluate

```
Get[".../verify_trace_euclidean_v9.m"]
```

in Mathematica. The `.m` file deliberately does not call `Exit`; only `run_verification.wls` turns the recorded status into a batch exit code.

After a manuscript revision, the maintainer keeps the manuscript outside the Git repository and runs the private refresh pipeline with:

```
.\build_all.ps1 -ManuscriptPath 'D:\private\path\Trace-Euclidean-v9.tex'
```

The temporary source snapshot and compiled manuscript are written only to ignored private-work directories. The pipeline regenerates the extracted inputs, displayed manuscript numbering, result files, and this verification manual, then invokes the repository manuscript-exclusion check. The optional executable parameters may be used when Python, WolframScript, or TeX Live is installed elsewhere. No network connection or third-party Wolfram package is required for the calculation run.